

Unsupervised Human Presence Detection

from Indoor Sensor Signals

A comparison of Isolation Forest, MLP Autoencoder, and CNN Autoencoder

Nur Bakhavadinovich Taabaldiev

AI Lab Project | Sapienza University of Rome | Prof. D. Avola

The problem

- Knowing if a room is occupied is useful for energy saving, security, and smart buildings.
- Many of these cases cannot use cameras, for privacy or cost reasons.
- The usual solution is supervised learning, but it needs labeled occupied and empty data, which is expensive to collect.
- My idea: treat presence as an anomaly. An empty room is easy to record. Train only on empty data, then flag anything that does not look empty.

Research question

Can an unsupervised model learn what an empty room looks like well enough to detect human presence, and does reading the time order of the signal help?

The data

UCI Occupancy Detection dataset

- Office room, one reading per minute, about 16 days.
- 20,560 records after cleaning. 77% empty, 23% occupied.
- Two stray header rows were mixed inside the data and removed.
- I use only Light and CO2, the two signals that react most to presence.



Red shading = occupied. Light and CO2 clearly jump during presence.

Setup

Windowing

Cut the signal into 30-minute windows of shape (30 x 2), stride 5. A window is occupied if anyone was present during any minute of it.

Split by time, not random

Windows overlap, so a random split would leak data. First 70% of empty windows for training, next 15% validation, last 15% test. All occupied windows go to test.

Normalize from train only

Z-score using mean and std from the training set only, so no test information leaks in.

Threshold

Same rule for every model: 95th percentile of validation scores. This keeps the comparison fair.

The three models

1. Isolation Forest

classical baseline

Uses 10 summary features per window (mean, std, min, max, range of each signal). Isolates unusual points with few random cuts. No time order.

2. MLP Autoencoder

feedforward network

Flattens the window to 60 values. Learns to rebuild empty windows. High reconstruction error = anomaly. Time order is lost in the flatten.

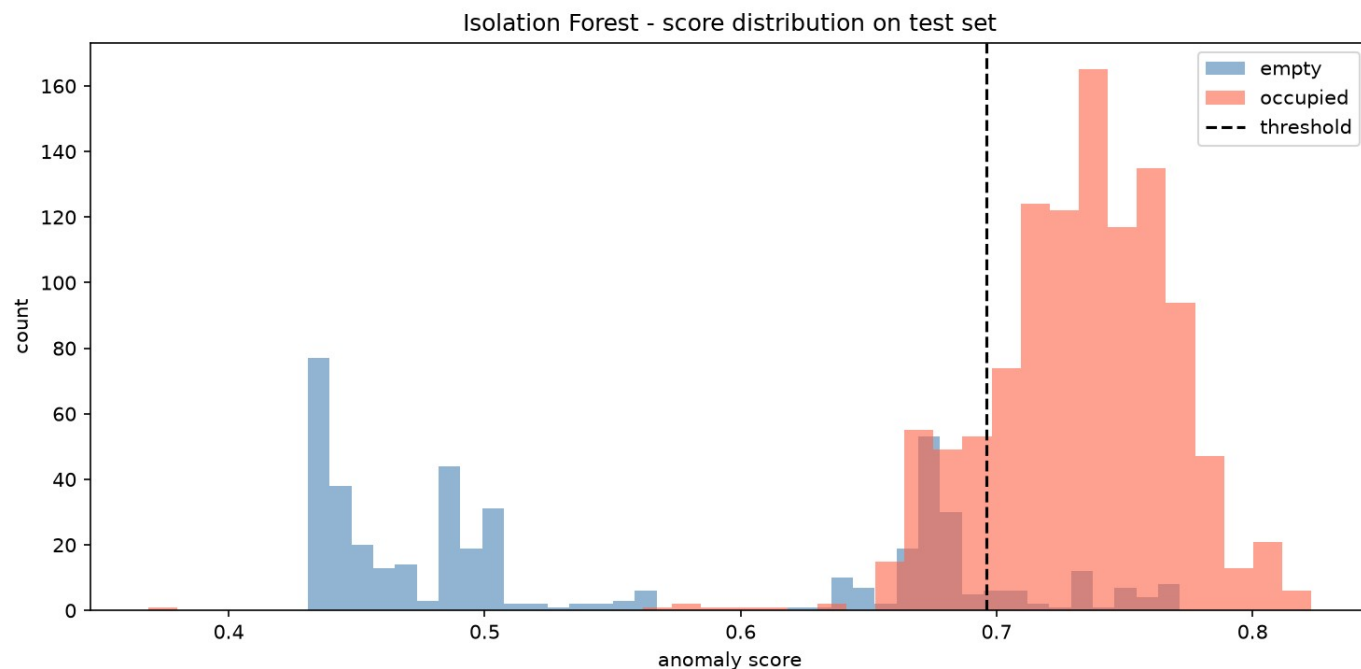
3. CNN Autoencoder

convolutional network

Keeps the window as a sequence of 2 channels over 30 steps. Conv filters read local patterns like a sharp CO2 rise. Time order is kept.

All three train only on empty windows. No occupied label is ever used in training.

Model 1: Isolation Forest



Empty and occupied scores separate clearly.

output

precision: 0.953

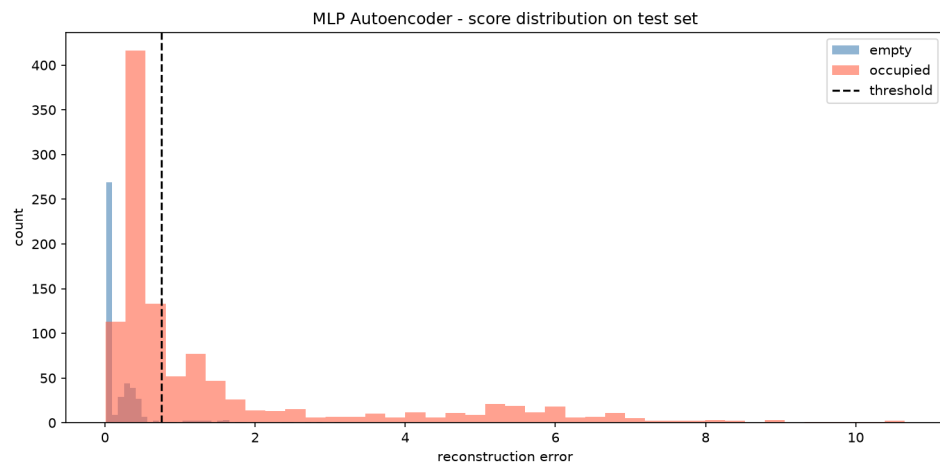
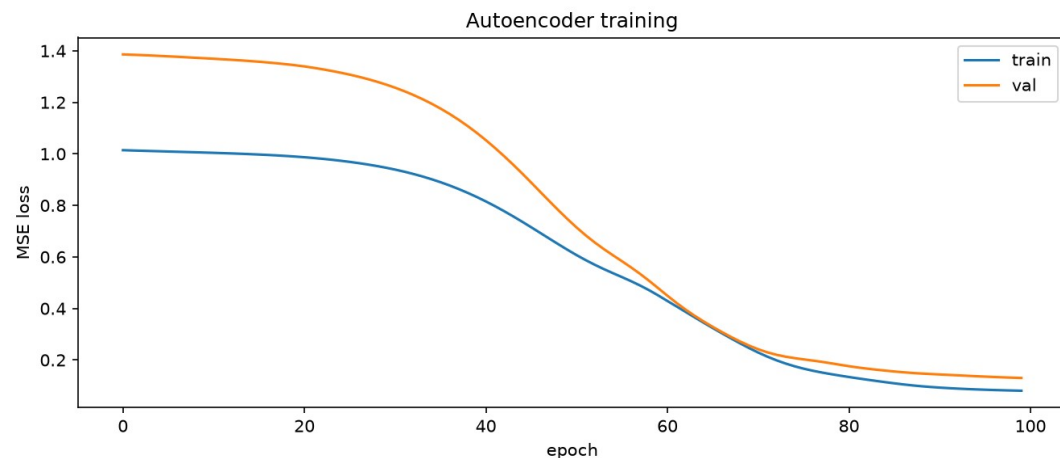
recall: 0.843

f1: 0.895

roc-auc: 0.928

Strong, balanced baseline. High precision, good recall. The number the neural nets must beat.

Model 2: MLP Autoencoder



output

precision: 0.948

recall: 0.399

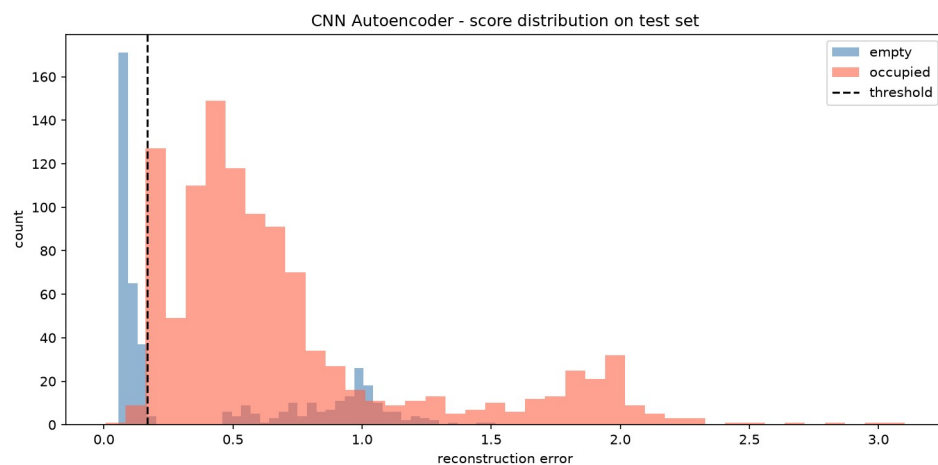
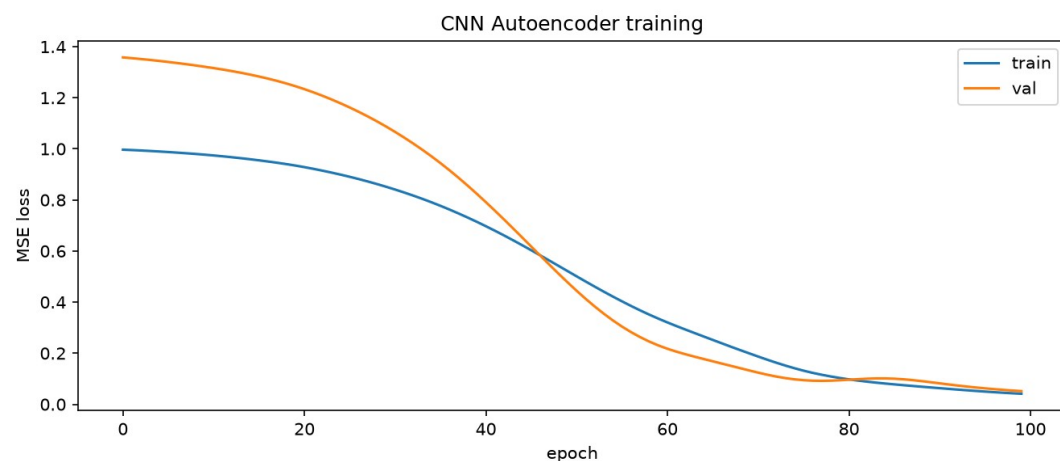
f1: 0.561

roc-auc: 0.884

Recall collapses to 0.40.

Training is healthy, no overfitting. But the model flattens the window and loses the time order. Many occupied windows have average values close to empty ones, so they get low reconstruction error and slip below the threshold.

Model 3: CNN Autoencoder



output

```
precision: 0.858  
recall:    0.988  
f1:        0.919  
roc-auc:   0.707
```

Best F1, recall almost 1.

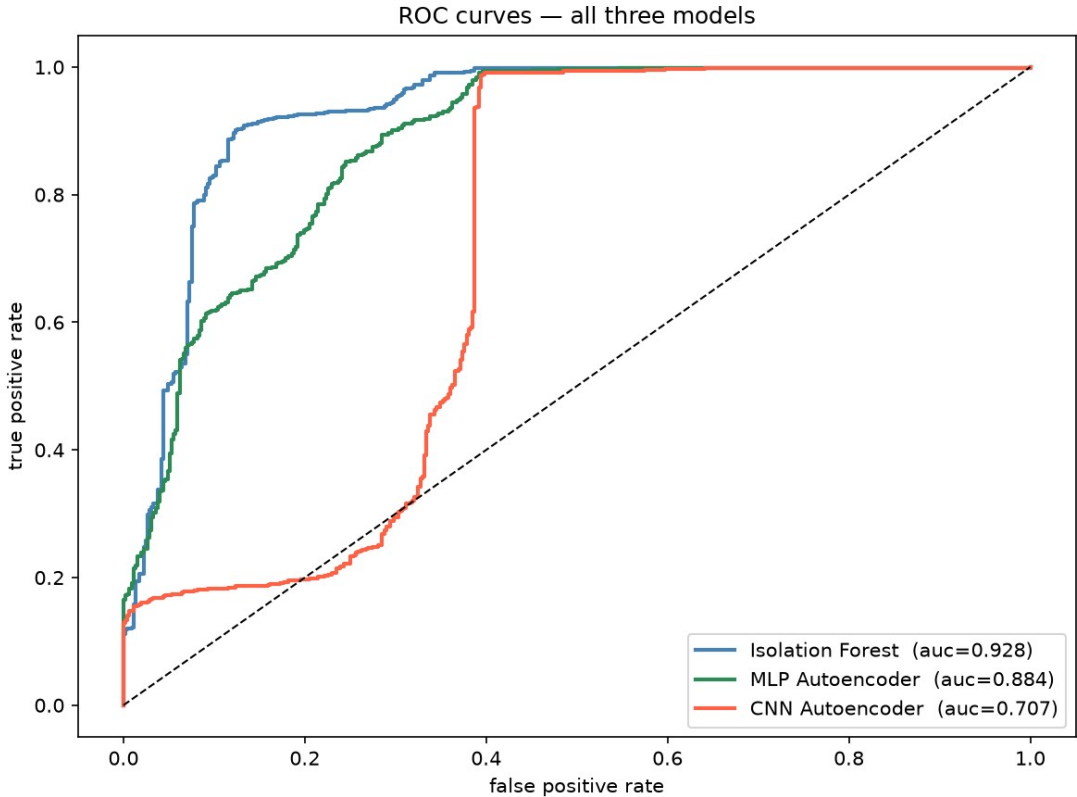
Keeping the sequence lets conv filters read local change. It misses only 13 occupied windows out of 1099. The cost is more false alarms and a lower ROC-AUC, because its scores sit in a narrow band.

Comparison

Model	Precision	Recall	F1	ROC-AUC
Isolation Forest	0.953	0.843	0.895	0.928
MLP Autoencoder	0.948	0.399	0.561	0.884
CNN Autoencoder	0.858	0.988	0.919	0.707

CNN wins on F1 and recall. Isolation Forest wins on ROC-AUC. MLP is weakest.

F1 depends on the chosen threshold. ROC-AUC is threshold independent. They can disagree, and here they do.



Why the results look like this

Why the MLP is weak. Flattening throws away the time order. The model learns the level of each signal but not its shape over 30 minutes, so occupied windows that start out looking calm are reconstructed too well.

Why the CNN helps. Convolution reads local change, so the model reacts to the dynamics of occupancy. Recall jumps to 0.988. But its scores are squeezed into a narrow range, so ROC-AUC drops.

Why the baseline stays strong. Ten simple features already capture most of what separates occupied from empty. A simple, well-chosen representation can match a neural network on a problem this size.

Practical takeaway. *If missing a person is costly (security), pick the CNN for high recall. If false alarms are costly (energy management), pick the Isolation Forest.*

Conclusion

- Presence detection works as an unsupervised, one-class problem. No occupied labels needed in training.
- Reading the time order matters. The CNN gets the best F1 (0.919) and recall (0.988).
- A simple classical baseline stays competitive and is better calibrated (ROC-AUC 0.928).
- No single model wins on every metric. The right choice depends on the cost of each error.

Future work. *Add a Transformer encoder. Test across different rooms. Try an adaptive threshold. Add the weaker signals and see if they help.*

Thank you

Nur Bakhavadinovich Taabaldiev

github.com/Nur2424/wifi-presence-detection